MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY
ALLAHABAD
PRAYAGRAJ - 211004

SMARTPARK AI

SMART PARKING SPACE DETECTION

A Technical Project Report

Submitted by:
Manik Dev Azad (20233178)
Roshna Rajesh (20233551)
Prakhar Patni (20233545)
Rudra Pratap Singh (20233552)
Department: Computer Science and Engineering
Institution: Motilal Nehru National Institute of Technology

Project repository: https://github.com/roshna18/SmartParkAI

Academic Year: 2026–2027

Table of Contents

S.No	Page Numbers	Content
1	3	Abstract
2	4	Introduction
3	5	Problem Definition and Objectives
4	6	Scope and Requirements
5	7	System Design and Architecture
6	8	Methodology and Detection Pipeline
7	9	Implementation Details
8	10	API Design and Data Representation
9	11	Testing, Reliability and Security
10	12	Results, Evaluation and Limitations
11	13	Conclusion and Future Work
12	14	References

Abstract

SmartPark AI is a computer-vision-based parking occupancy detection application. It accepts an uploaded image of a parking facility, detects individual parking bays using a configured Ultralytics YOLO model, maps model labels to application-level states, calculates occupancy statistics, and returns an annotated image with structured detection data. The application uses a React 19 and Vite frontend connected to a Python FastAPI backend. OpenCV is used for image decoding, annotation rendering, and image encoding, while Pydantic schemas structure and validate application data.

The project focuses on turning a parking-lot image into interpretable outputs: the total number of detected bays, vacant and occupied counts, unknown classifications, occupancy percentage, confidence scores, bounding-box coordinates, and a color-coded visual overlay. The backend is organized into API routes, services, model inference, image-processing utilities, schemas, and core infrastructure. The repository also documents local JSON persistence for parking-space layouts, rate limiting, validation, atomic file writes, error handling, and automated tests.

This report describes the system as implemented in the public repository. It does not assume continuous live CCTV streaming, deployment at a real parking facility, or a verified accuracy percentage, because these are not established by the inspected project documentation. The system is best understood as an image-based prototype that demonstrates a modular full-stack workflow for parking-bay detection and occupancy analytics.

Keywords: Smart parking, computer vision, object detection, YOLO, FastAPI, React, OpenCV, occupancy classification.

1. Introduction

1.1 Background

Parking facilities contain multiple individual bays whose availability changes as vehicles arrive and leave. A parking manager or visitor may need a quick estimate of how many bays are free. Computer vision provides a way to infer parking-space status from images, avoiding the need to inspect each bay manually for every observation.

SmartPark AI applies an object-detection model to a parking image and presents the resulting predictions through a browser-based interface. Instead of only returning a count, the application also provides per-bay information and a visual overlay so users can inspect where the model found each space and how it classified it.

1.2 Project Overview

The repository describes a web application with a React 19 and Vite frontend and a FastAPI backend. The backend validates an uploaded image, decodes it into an image array, invokes the YOLO inference layer, computes occupancy metrics, draws annotations using OpenCV, and encodes the annotated result as a Base64 data URL. The frontend provides file upload and preview, loading and error states, metric cards, and a table of individual predictions.

1.3 Motivation

The project demonstrates how a machine-learning model can be integrated into a practical software system. Its contribution is not only the inference call: it includes input validation, modular service boundaries, structured response schemas, visual explanation of detections, and reliability controls around a compute-intensive endpoint.

1.4 Report Organisation

The report first defines the problem and project scope, then describes the architecture and inference pipeline. Subsequent chapters discuss implementation, API design, testing and reliability, followed by an evidence-based evaluation, limitations, and possible future work.

2. Problem Definition and Objectives

2.1 Problem Statement

Given a supported image of a parking facility, identify visible parking bays and classify each detected bay as vacant, occupied, or unknown. Present the detections in a way that allows a user to inspect the predicted state and understand the overall occupancy of the depicted lot.

2.2 Objectives

- Accept a parking-lot image through a browser interface.
- Validate the uploaded image's extension, size, and decodability before inference.
- Use the configured YOLO model to generate parking-bay detections.
- Map model labels to application statuses: vacant, occupied, or unknown.
- Calculate total detections, state counts, and occupancy percentage.
- Draw colored bounding boxes and status labels on the input image.
- Return structured data suitable for dashboard metrics and a per-bay table.
- Organize the implementation into loosely coupled frontend and backend modules.
- Provide reliability mechanisms such as rate limiting, schema validation, safe persistence, and error handling.

2.3 Intended Users and Use Case

The interface is suitable for a user who has a parking-lot image and wants a quick, visual estimate of occupancy. The repository does not establish a particular commercial deployment, user study, or production parking-site integration. The main use case is therefore image-based inspection and demonstration.

2.4 Success Criteria

Area	Evidence of functional success
Input handling	Supported image is accepted; unsupported, oversized, empty, or corrupted input is rejected with an error.
Detection	The model returns parking-bay bounding boxes and confidence values.
Classification	Each detection is assigned vacant, occupied, or unknown based on its model label.
Analytics	The response includes counts and an occupancy percentage.
Visual output	The output image shows status-specific annotations.
Interface	The UI displays summary metrics and per-bay results.
Reliability	Validation, rate limiting, safe writes, and automated tests are provided in the codebase.

3. Scope and Requirements

3.1 Functional Requirements

ID	Requirement	Implementation location / evidence
FR-01	Upload a parking image	Frontend FileUpload component and POST /api/analyze.
FR-02	Validate and decode the image	Image-processing encoder utility.
FR-03	Run parking-bay detection	ML inference module using the configured YOLO model.
FR-04	Classify and annotate detections	Label classification and OpenCV annotation utilities.
FR-05	Compute occupancy statistics	Analysis service calculates total, vacant, occupied, unknown, and percentage.
FR-06	Display per-bay results	ResultsTable displays status, confidence, and bounding coordinates.
FR-07	Display summary KPIs	StatsPanel shows total spaces, vacant, occupied, and occupancy.
FR-08	Save/read parking layouts	GET /api/spaces and POST /api/spaces endpoints.
FR-09	Handle failures gracefully	Centralized errors, frontend error UI, and loading states.

3.2 Non-Functional Requirements

- Maintainability: separate routes, services, schemas, ML logic, and image-processing utilities.
- Input safety: enforce supported file extensions and a maximum upload size documented as 15 MB by default.
- Reliability: use atomic file replacement for parking-space persistence and protect inference with a lock.
- Operational control: configurable CORS origins, rate limiting, logging, and environment-based settings.
- Usability: show a preview, loading feedback, errors, color-coded annotations, and structured results.
- Testability: backend unit/integration tests and frontend tests are documented in the repository.

3.3 Scope Boundaries

The current workflow is based on an uploaded image. The UI text references surveillance or aerial imagery, but the inspected implementation does not establish continuous live video ingestion, automatic camera connectivity, vehicle tracking over time, parking reservations, payment processing, or mobile-app functionality. These features should be treated as potential extensions rather than existing capabilities.

4. System Design and Architecture

The system follows a layered design. The frontend handles user interaction and rendering; the API layer accepts requests and applies request-level controls; the analysis service coordinates image validation, inference, metrics, and rendering; and specialized modules handle model loading, predictions, annotations, encoding, schemas, and persistence.

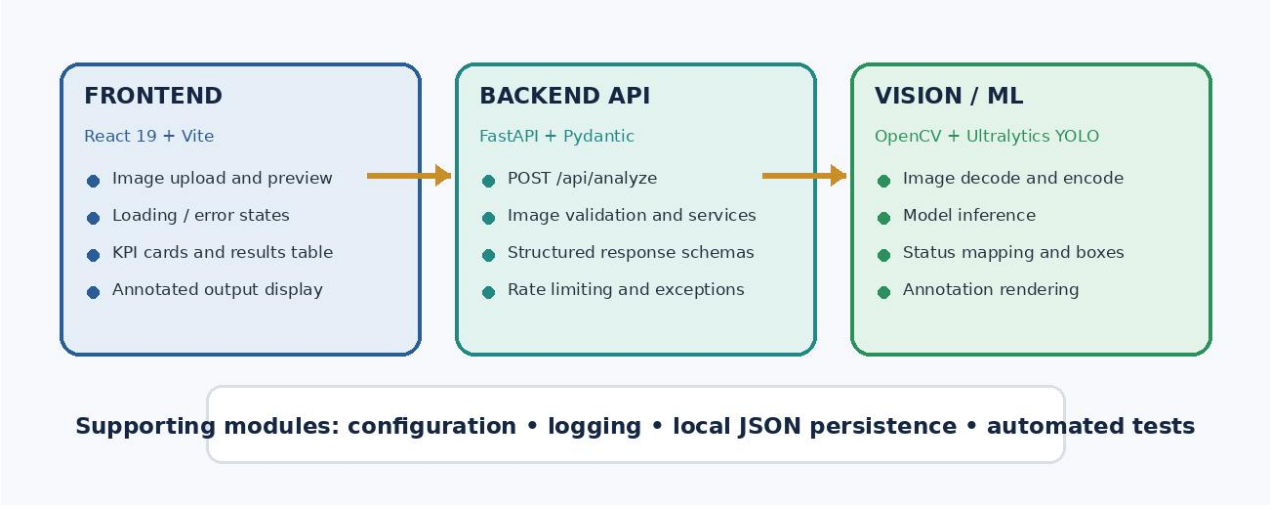


Figure 4.1. High-level architecture of SmartPark AI, based on the repository structure.

4.1 Frontend Layer

The frontend is implemented with React 19 and Vite. App.jsx composes the main page and uses the useAnalysis hook to manage the selected file, preview, result, loading state, errors, analysis request, and reset behaviour. FileUpload handles file selection and initiating analysis. AnalysisResults presents the returned results, while StatsPanel and ResultsTable show summary and per-bay information. Common components provide loading and error feedback.

4.2 Backend API Layer

FastAPI exposes the analysis route and routes for parking-space layouts. The analysis endpoint accepts an uploaded file and checks the configured rate limiter before delegating work to the analysis service. Pydantic models describe the request and response payloads. Configuration, exceptions, logging, and rate limiting are kept in core modules.

4.3 Machine-Learning and Image Layer

The inference module obtains the model through a thread-safe model loader and invokes prediction with configured confidence and image-size parameters. Each model box is converted into a structured detection containing an identifier, label, normalized application status, confidence, and bounding-box coordinates. OpenCV utilities render the boxes and labels and encode the resulting image for the frontend.

4.4 Persistence Layer

Parking-space layout data is stored locally in JSON according to the repository documentation. The spaces service validates the supplied layout and writes through a temporary file before replacing the target file atomically. A reference parking-lot image upload route is also documented.

5. Methodology and Detection Pipeline

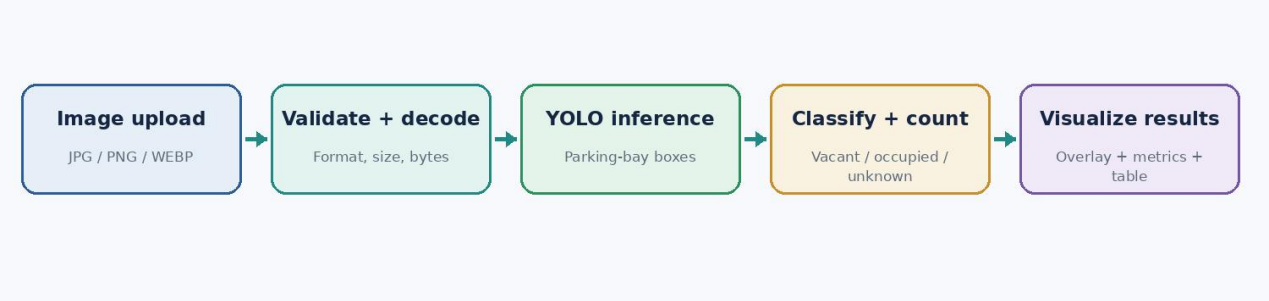


Figure 5.1. Image-analysis sequence implemented by the backend.

5.1 Image Upload and Validation

The frontend sends an image to the analysis API. The backend accepts JPG, JPEG, PNG, and WEBP extensions, reads at most the configured maximum size plus one byte to detect oversize uploads, rejects empty content, and uses OpenCV to decode the image.

5.2 Model Inference

The configured model repository is simahanyan/parking-lot-yolo26m, with the weight path configured as weights/parking_lot.pt. The default confidence threshold is 0.25, and the default inference image size is 1280; both are configurable through environment settings.

5.3 Status Mapping

The application maps raw model labels to three statuses. Labels containing terms such as “empty”, “vacant”, or “free” map to vacant. Labels containing terms such as “occupied”, “filled”, or “taken” map to occupied. Labels that match neither group are assigned unknown.

Application status	Overlay colour	Meaning
Vacant	Green	Model label indicates an available / empty bay.
Occupied	Red	Model label indicates a bay that is taken.
Unknown	Grey	The raw label does not match the configured vacant/occupied keywords.

5.4 Metrics

The analysis service computes total detections, vacant count, occupied count, and unknown count. Occupancy percentage is computed from classified bays only:

$$\text{Occupancy (\%)} = \text{Occupied} / (\text{Occupied} + \text{Vacant}) \times 100$$

If no bays are classified as vacant or occupied, the percentage is set to 0.0. Unknown detections are excluded from the denominator so that uncertain predictions do not inflate the occupancy rate.

5.5 Annotation and Response

OpenCV draws a rectangle and status text for each detection. Vacant boxes are green, occupied boxes are red, and unknown boxes are grey. The annotated image is encoded as JPEG and returned as a Base64 data URL along with the structured detections, metrics, image dimensions, an availability flag, and a note that model predictions may vary with image conditions.

6. Implementation Details

6.1 Backend Modules

Module	Responsibility
app/api/routes/analyze.py	Defines POST /analyze and applies rate limiting before analysis.
app/services/analysis_service.py	Orchestrates validation, inference, metrics, annotation, encoding, and response creation.
app/ml/inference.py	Runs YOLO prediction and converts boxes into DetectionItem records.
app/ml/model_loader.py	Manages model loading; documented as thread-safe and lazy.
app/image_processing/encoder.py	Validates and decodes uploaded images; encodes annotated images as Base64 JPEG data URLs.
app/image_processing/annotations.py	Maps labels to statuses and draws color-coded boxes and labels.
app/api/routes/spaces.py	Provides GET/POST /spaces and POST /lot/upload.
app/services/spaces_service.py	Handles layout persistence and atomic file writes.
app/schemas/	Defines typed request and response structures.
app/core/	Contains settings, domain exceptions, logging, and rate limiting.

6.2 Frontend Modules

Component / module	Responsibility
App.jsx	Composes the page and connects the upload, analysis results, loading, and error components.
useAnalysis.js	Encapsulates UI state and the analysis request lifecycle, including cancellation of stale requests.
FileUpload.jsx	Provides file selection, drag-and-drop upload interaction, preview, and analyze/reset controls.
AnalysisResults.jsx	Composes the analysis result view.
StatsPanel.jsx	Displays total spaces, vacant count, occupied count, and occupancy percentage.
ResultsTable.jsx	Displays each detection's ID, status, confidence, and bounding-box coordinates.
services/api.js	Centralizes frontend HTTP requests.
utils/formatting.js	Provides formatting and status-badge helpers.

6.3 Configuration

Setting	Default documented value	Purpose
MODEL_REPO	simahanyan/parking-lot-yolo26m	Model repository identifier.
MODEL_FILENAME	weights/parking_lot.pt	Configured model weights path.
MODEL_CONFIDENCE_THRESHOLD	0.25	Confidence threshold for detections.
MODEL_IMAGE_SIZE	1280	Image size passed to YOLO inference.

MAX_FILE_SIZE_BYTES	15 MB	Maximum accepted image size.
RATE_LIMIT_REQUESTS	30	Request limit in a 60-second window by default.
VITE_API_URL	http://127.0.0.1:8000	Frontend API base URL.

7. API Design and Data Representation

7.1 Main Analysis Endpoint

Property	Description
Method and route	POST /api/analyze (the route is mounted under the API prefix).
Request	Multipart form upload with an image file.
Processing	Rate limit → image validation/decode → YOLO inference → metrics → annotations.
Response	Message, total spaces, occupied, vacant, unknown, occupancy percentage, detection arrays, annotated image data URL, availability flag, note, and image dimensions.

7.2 Per-Detection Representation

Each detection is represented with a generated identifier (for example, P1), the original model label, the application status, a Boolean occupied field, a confidence value, and bounding-box coordinates in [x1, y1, x2, y2] order. Coordinates are integer pixel positions constrained to the input image bounds.

7.3 Parking-Space Layout Endpoints

Endpoint	Purpose
GET /api/spaces	Returns the saved parking-space layout.
POST /api/spaces	Validates and persists a submitted parking-space layout.
POST /api/lot/upload	Accepts a reference parking-lot image and saves it locally.

7.4 Response and Error Handling

The backend uses response schemas to keep outputs structured. Unsupported extensions, empty files, oversized files, and corrupted image bytes are rejected before model inference. Centralized exception handling is intended to prevent internal tracebacks or system paths from being exposed to clients. The frontend has dedicated loading and error components, and its analysis hook cancels stale requests when a newer request supersedes them.

8. Testing, Reliability and Security

8.1 Automated Testing Strategy

The repository documents backend tests using pytest and frontend tests using Vitest with Testing Library. The README describes the following coverage areas:

- Schema validation and rejection of malformed payloads.
- Parking-space persistence and atomic file writes.
- Image decoding, validation, and Base64 encoding.
- Detection annotation rendering.
- Mocked model inference and error handling.
- API route integration for health, spaces, and analysis endpoints.
- Frontend utility formatting, KPI rendering, results-table behaviour, and empty states.

8.2 Reliability Controls

Control	Purpose
Input validation	Reject unsupported, oversized, empty, or corrupted images.
Rate limiting	Limit repeated requests to the compute-heavy analysis endpoint.
Thread-safe model management	Avoid conflicting model loading or inference operations across requests.
Atomic persistence	Write layout data to a temporary file before replacing the saved JSON file.
Centralized exception handling	Return controlled errors rather than leaking internal details.
Request cancellation	Prevent stale frontend requests from overriding newer analysis state.
Structured logging	Record useful application events for debugging and operation.

8.3 Security Considerations

The project includes configurable CORS origins, request limiting, schema validation, and controlled exception handling. These controls reduce some common operational risks but do not, by themselves, constitute a complete production security assessment. Before deployment on a public network, the application should be reviewed for authentication and authorization needs, storage permissions, request-size enforcement at the reverse proxy, secrets handling, logging of sensitive information, dependency updates, and multi-worker rate-limit behaviour.

8.4 Reproducibility

The README documents a local setup using Python 3.10 or higher and Node.js 18 or higher (20+ recommended), plus a Docker Compose option. It also documents commands for running backend and frontend tests. The actual outcome of those tests is not asserted in this report; test coverage described in a README is not the same as a recorded successful test run.

9. Results, Evaluation and Limitations

9.1 Functional Output

Based on the inspected implementation, a successful analysis response is designed to provide: (i) the total number of detected bays, (ii) counts of vacant, occupied, and unknown detections, (iii) an occupancy percentage, (iv) individual detection records with confidence and coordinates, and (v) an annotated image. The frontend displays the summary and the individual predictions.

9.2 Evaluation Status

The repository README does not provide a benchmark dataset, a confusion matrix, precision, recall, F1-score, mean average precision, measured latency, or a verified end-to-end accuracy figure. Consequently, no numerical performance claim is made here. The report describes implemented behaviour and the intended output format rather than claiming a measured level of model performance.

9.3 Suggested Evaluation Protocol

To assess the detector scientifically, the next evaluation should use a labelled test set representative of the intended camera viewpoints, lighting, weather, bay layouts, and vehicle types. Ground-truth boxes and occupancy labels should be prepared independently of the model outputs. Report at least:

- Detection precision and recall, plus mAP at a clearly stated IoU threshold.
- Vacant/occupied classification precision, recall, F1-score, and confusion matrix.
- Occupancy percentage error against manually labelled ground truth.
- Median and 95th-percentile inference latency on specified hardware.
- Failure cases grouped by occlusion, camera angle, shadows, image quality, and unusual layouts.

9.4 Current Limitations

- Image-based workflow: continuous live CCTV processing is not established by the inspected implementation.
- Model-dependent predictions: performance may vary with viewpoint, lighting, occlusion, and layout.
- Unknown state: labels outside the configured keyword groups are marked unknown.
- No published benchmark: quantitative accuracy and latency are not provided in the README.
- Local persistence: parking layouts are stored in local JSON rather than a database-backed multi-user system.
- No temporal tracking: the inspected workflow analyses an image and does not establish tracking of individual vehicles over time.
- No reservation/payment workflow: the scope is detection and occupancy analytics.

10. Conclusion and Future Work

10.1 Conclusion

SmartPark AI demonstrates an end-to-end software workflow for image-based parking occupancy detection. A React/Vite interface submits an image to a modular FastAPI backend; the backend validates and decodes it, runs a configured YOLO detector, maps model labels into occupancy states, computes summary metrics, draws annotations, and returns structured results. The UI then presents the annotated image, KPI cards, and a per-bay table.

The repository's engineering structure is a notable aspect of the project: inference, image processing, API routing, schemas, configuration, persistence, and UI components are separated into modules. Validation, rate limiting, atomic layout writes, thread-safe inference, error handling, and documented automated tests support maintainability and robustness. At the same time, an academic evaluation should distinguish implemented functionality from unmeasured performance. A representative labelled benchmark is required before making accuracy claims.

10.2 Future Work

- Build a labelled evaluation dataset and publish reproducible detection and classification metrics.
- Add an optional video/camera ingestion layer if real-time monitoring is a project requirement.
- Introduce temporal smoothing or tracking to reduce flicker between consecutive video frames.
- Extend unknown-label handling with explicit model-class configuration and diagnostics.
- Add user authentication, authorization, audit logs, and production-grade rate limiting if deployed publicly.
- Move persistent layouts to a managed database if multi-user operation or history is required.
- Measure runtime and memory usage on clearly specified CPU/GPU hardware.
- Improve accessibility and usability through camera-angle guidance, image-quality feedback, and clearer explanation of uncertain predictions.

References

[1] roshna18, “SmartParkAI: SmartPark AI,” GitHub repository. Available: https://github.com/roshna18/SmartParkAI (accessed 28 October 2026).

[2] Ultralytics, “Ultralytics YOLO Documentation.” Available: https://docs.ultralytics.com/ (accessed 28 October 2026).

[3] FastAPI, “FastAPI Documentation.” Available: https://fastapi.tiangolo.com/ (accessed 28 October 2026).

[4] React, “React Documentation.” Available: https://react.dev/ (accessed 28 October 2026).

[5] OpenCV, “OpenCV Documentation.” Available: https://docs.opencv.org/ (accessed 28 October 2026).

[6] Pydantic, “Pydantic Documentation.” Available: https://docs.pydantic.dev/ (accessed 28 October 2026).

Repository-specific implementation details in this report are based on the README and inspected source files, including the analysis route, analysis service, inference module, image-processing utilities, configuration, and frontend components.